Yahya Ayyash Ashdaqi

5025231210

[E03] NestJS CRUD

Setup

Instalasi NestJS CLI

```
npm install -g @nestjs/cli
```

Buat project baru

```
nest new .
```

instalasi dependensi

```
npm install
```

instalasi depedensi tambahan

```
npm add class-validator class-transformer
```

Instalasi depedensi testing

```
npm add -D @nestjs/testing supertest
```

Do

Generate user resource

```
nest generate resource users
```

1. Membuat scaffold lengkap dengan resource users
2. Hasil generate:

```
src/users/
├── dto/
│   ├── create-user.dto.ts      DTO untuk input pembuatan user
│   └── update-user.dto.ts      DTO untuk input update user
├── entities/
│   └── user.entity.ts          Entity user
├── users.controller.ts         Controller (handle request HTTP)
├── users.module.ts             Module (mendaftarkan controller & service)
├── users.service.ts            Service (business logic)
```

Implementasikan layanan users di file yang telah di-generate di dalam folder `src/users/`

Tambah state penyimpanan sementara

```
private users: User[] = [];
private nextId = 1;
```

- Karena tidak menggunakan database, disini kita simpan user di dalam array.
- NextId dipakai untuk increment user baru.

Ubah create()

```
create(createUserDto: CreateUserDto): User {
  const newUser: User = {
    id: this.nextId++,
    ...createUserDto,
  };
  this.users.push(newUser);
```

```
  return newUser;
}
```

Ubah findOne()

- Menggunakan `...createUserDto` agar semua field DTO masuk otomatis, kemudian simpan user baru ke dalam array user

```
findOne(id: number): User {
  const user = this.users.find((u) => u.id === id);
  if (!user) {
    throw new NotFoundException(`User with ID ${id} not found`);
  }
  return user;
}
```

Ubah update()

- Cari user berdasrakan ID

```
update(id: number, updateUserDto: UpdateUserDto): User {
  const user = this.findOne(id); // akan lempar error kalau tidak ada
  Object.assign(user, updateUserDto);
  return user;
}
```

- Memanggil `findOne()` untuk memastikan user ada
- Objek.assign() untuk menimpa field lama dengan field baru dari updateUserDTO

Ubah remove()

```
remove(id: number): void {
  const index = this.users.findIndex((u) => u.id === id);
  if (index === -1) {
    throw new NotFoundException(`User with ID ${id} not found`);
  }
  this.users.splice(index, 1);
}
```

- Cari index user
- Kalau tidak ada akan error, dan jika ada akan di hapus menggunakan splice

Melakukan beberapa perubahan pada file `user.entity.ts`, `create-user.dto.ts`, dan `update-user.dto.ts`

Entity

```
export class User {
  id: number;
  name: string;
  email: string;
}
```

- Entitiy mewakili struktur data / model domain yang akan dipetakan ke database. Field `id`, `name`, dan `email` mempresentasikan kolom pada tabel `users`

Create DTO

```
import { IsEmail, IsNotEmpty } from 'class-validator';

export class CreateUserDto {
  @IsNotEmpty()
  name: string;

  @IsEmail()
  email: string;
}
```

- DTO (Data Transfer Object) adalah format data yang diterima client. `CreateUserDto` dipakai ketika membuat user baru `POST/users`

Update DTO

```
import { PartialType } from '@nestjs/mapped-types';
import { CreateUserDto } from './create-user.dto';

export class UpdateUserDto extends PartialType(CreateUserDto) {}
```

- Tidak perlu semua field diisi saat update. Misal client cuma ingin update name. Hasilnya, `UpdateUserDto` sama seperti `CreateUserDto`, tapi semua property optional.

Melakukan update pada file `src/main.ts`

```
import { ValidationPipe } from '@nestjs/common';

app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
  }),
);
```

- `ValidationPipe` => Untuk melakukan validasi dan transformasi data request
- `whitelist : true` => Menghapus properti yang tidak ada di DTO

```
{
  "name": "Yahya",
  "email": "yahya@example.com",
  "age": 25   // <-- property ini akan dihapus karena tidak ada di DTO
}
```

- `transform: true` => Mengubah data plain JSON dari request menjadi instance class DTO.
- `forbidNonWhitelisted: true` => ika ada property asing yang tidak ada di DTO, akan dilempar error 400 Bad Request.

```
{
  "name": "Yahya",
  "email": "yahya@example.com",
  "age": 25   // <-- property ini menyebabkan error
}
```

Run development server

```
npm run start:dev
```

- Menjalankan NestJS

Run end-to-end (e2e) test

```
pnpm test:e2e
```

- Menjalankan semua e2e tests untuk memverifikasi aplikasi berjalan sesuai yang di inginkan.
- Menguji alur penuh dari request ⟶ controller ⟶ service ⟶ response.

Perbandingan pure NodeJS

Struktur kode

NestJS

- Memisahkan controller, service, DTO, entitiy => Kode lebih rapi dan modular

NodeJS

- Semua logic di gabung di dalam satu file => Berantakan

Validasi dan transformasi

NestJS

- `ValidationPipe + class-validator` otomatis memvalidasi input dan mengubah JSON menjadi class instance.

NodeJS

- Validasi input harus manual `(if (!req.body.name) …)` dan transformasi object juga manual.

Testing

NestJS

- Integrasi e2e test `(@nestjs/testing + supertest)` sudah mudah di-setup.

NodeJS

- Harus setup testing environment manual ⟶ lebih banyak konfigurasi.

Kesimpulan

- Dari sisi implementasi, NestJS lebih baik karena mengurangi boilerplate, membuat kode modular, mempermudah validasi, transformasi, dan testing.